Assignment 7: Expression Evaluation using Stack

C Program (Postfix expression evaluation using an array-based stack; Push Operand, Pop Operands, Apply Operator, Display Result)

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

int isFull();
int isEmpty();
void push(int);
int pop();
int peek();

int isOperator(char *token);
int power(int base, int exponent);
int applyOperator(int a, int b, char op, int *valid);

int isFull()
{
    if (top == MAX - 1)
        return 1;
    else
        return 0;
}

int isEmpty()
{
    if (top == -1)
        return 1;

    else
        return 0;
}

void push(int item)
{
    if (isFull())
    {
        printf("Stack Overflow\n");
    }
    else
    {
        top = top + 1;
        stack[top] = item;
    }
}

int pop()
{
    int item;
```

```
    if (isEmpty())
    {
        printf("Stack Underflow\n");
        return 0;
    }
    else
    {
        item = stack[top];
        top = top - 1;
        return item;
    }
}

int peek()
{
    if (isEmpty())
        return 0;
    else
        return stack[top];
}

int isOperator(char *token)
{
    if (strlen(token) == 1 &&
        (token[0] == '+' || token[0] == '-' ||
         token[0] == '*' || token[0] == '/' ||
         token[0] == '%' || token[0] == '^'))
        return 1;

    return 0;
}

int power(int base, int exponent)
{
    int result = 1;
    int i;

    for (i = 0; i < exponent; i++)
        result = result * base;

    return result;
}

int applyOperator(int a, int b, char op, int *valid)
{
    *valid = 1;

    switch (op)
    {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;

        case '/':
            if (b == 0)
            {
                *valid = 0;
                printf("Error: Division by zero\n");
                return 0;
            }
            return a / b;
```

```
        case '%':
            if (b == 0)
            {
                *valid = 0;
                printf("Error: Modulo by zero\n");
                return 0;
            }
            return a % b;

        case '^':
            if (b < 0)
            {
                *valid = 0;
                printf("Error: Negative exponent is not supported\n");
                return 0;
            }
            return power(a, b);

        default:
            *valid = 0;
            return 0;
    }
}

int evaluatePostfix(char expression[], int *valid)
{
    char *token;
    char copy[MAX];
    char *endptr;
    long number;
    int operand1, operand2, result;

    top = -1;
    *valid = 1;

    strcpy(copy, expression);

    token = strtok(copy, " \t\n");

    while (token != NULL)
    {

        number = strtol(token, &endptr, 10);

        if (*token != '\0' && *endptr == '\0')
        {
            if (number < -2147483648L || number > 2147483647L)
            {
                printf("Error: Operand out of integer range\n");
                *valid = 0;
                return 0;
            }

            push((int)number);
        }
        else if (isOperator(token))
        {

            if (top < 1)
            {
```

```
                printf("Error: Invalid postfix expression\n");
                *valid = 0;
                return 0;
            }

            operand2 = pop();

            operand1 = pop();

            result = applyOperator(operand1, operand2, token[0], valid);

            if (!(*valid))
                return 0;

            push(result);
        }
        else
        {
            printf("Error: Invalid token '%s'\n", token);
            *valid = 0;
            return 0;
        }

        token = strtok(NULL, " \t\n");
    }

    if (top == 0)
        return pop();

    printf("Error: Invalid postfix expression\n");
    *valid = 0;
    return 0;
}

int main()
{
    char expression[MAX];
    int result;
    int valid;

    printf("===== Postfix Expression Evaluation using Stack =====\n");
    printf("Supported operators: +  -  *  /  %%  ^\n");
    printf("Enter postfix expression (use spaces between operands): ");

    fgets(expression, MAX, stdin);
    expression[strcspn(expression, "\n")] = '\0';

    result = evaluatePostfix(expression, &valid);

    if (valid)
    {
        printf("\nPostfix Expression : %s\n", expression);
        printf("Evaluated Value    : %d\n", result);
    }

    return 0;
}
```

Output

Terminal execution and sample results



